Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 06

Q-Learning for Path Finding in a Maze

Aim

To implement the Q-Learning reinforcement learning algorithm in which an agent learns action values from rewards in a grid maze, and then follows the learned Q-values to find a path from the start S to the goal G.

Code and Output

In [1]:
```
import numpy as np, random
random.seed(42); np.random.seed(42)
maze=["S....",".##..","...#.","##...","....G"]
ROWS,COLS=len(maze),len(maze[0]); start=(0,0); goal=(4,4)
states=[(r,c) for r in range(ROWS) for c in range(COLS) if maze[r][c]!="#"]
state_id={s:i for i,s in enumerate(states)}
actions=[(-1,0),(1,0),(0,-1),(0,1)]
action_names=["Up","Down","Left","Right"]
Q=np.zeros((len(states),len(actions)))
alpha,gamma,epsilon=.2,.95,.25
def step(s,a):
    dr,dc=actions[a]; ns=(s[0]+dr,s[1]+dc)
    if not (0<=ns[0]<ROWS and 0<=ns[1]<COLS) or maze[ns[0]][ns[1]]=="#":
        ns=s
    done=ns==goal
    return ns,(100 if done else -1),done
episodes=5000
for ep in range(episodes):
    s=start
    for _ in range(100):
        if random.random()<epsilon: a=random.randrange(4)
        else: a=int(np.argmax(Q[state_id[s]]))
        ns,reward,done=step(s,a)
        target=reward+gamma*(0 if done else np.max(Q[state_id[ns]]))
        Q[state_id[s],a]+=alpha*(target-Q[state_id[s],a])
        s=ns
        if done: break
    epsilon=max(.02,epsilon*.999)
print("Training episodes:",episodes)
print("Q-table shape:",Q.shape)
```

```
Training episodes: 5000
Q-table shape: (20, 4)
```

In [2]:
```
s=start; path=[s]
for _ in range(40):
```

```
    if s==goal: break
    # Select a valid move with the highest learned Q-value.
    candidates=[]
    for a in range(4):
        ns,_,_=step(s,a)
        if ns!=s: candidates.append((Q[state_id[s],a],ns))
    if not candidates: break
    s=max(candidates,key=lambda item:item[0])[1]
    path.append(s)
    if s==goal: break
print("Maze:")
for row in maze: print(row)
print("Learned path:", " -> ".join(map(str,path)))
print("Reached goal:",s==goal,"| Steps:",len(path)-1)
```

```
Maze:
S....
.##..
...#.
##...
....G
Learned path: (0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (1, 3) -> (1, 4) -> (2, 4)
-> (3, 4) -> (4, 4)
Reached goal: True | Steps: 8
```